

CPSC 483 – Homework 2

Name: Jonathan Alvarado

Professor: Nathan Justin

Collaborators: None

Exercise 1 – Confusion Matrix

Going through the 20 transactions, with "Fraud" as the positive label:

- **True Positives (TP):** predicted Fraud and it was Fraud. That's transactions 1 and 2, so 2.
- **False Negatives (FN):** predicted Not Fraud but it was Fraud. That's transaction 3, so 1.
- **False Positives (FP):** predicted Fraud but it was Not Fraud. That's transactions 4 and 5, so 2.
- **True Negatives (TN):** predicted Not Fraud and it was Not Fraud. That's transactions 6–20, so 15.

	Predicted Fraud	Predicted Not Fraud
Actual Fraud	TP = 2	FN = 1
Actual Not Fraud	FP = 2	TN = 15

Check: $2 + 1 + 2 + 15 = 20$ transactions.

Exercise 2 – Metrics

(a) Accuracy

$$\text{Accuracy} = (\text{TP} + \text{TN}) / \text{Total} = (2 + 15) / 20 = 17/20 = 0.85 = 85\%$$

The model correctly predicted either fraud or not fraud on 85% of all transactions in the test set.

(b) Error

$$\text{Error} = 1 - \text{Accuracy} = (\text{FP} + \text{FN}) / \text{Total} = (2 + 1) / 20 = 0.15 = 15\%$$

The model got 15% of transactions wrong. It either flagged a normal transaction as fraud or missed an actual fraud.

(c) Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) = 2 / (2 + 2) = 0.50 = 50\%$$

Of all the transactions the model flagged as fraud, only 50% were actually fraud. Half the time it flags something, it's a false alarm on a real customer's normal purchase.

(d) Recall

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) = 2 / (2 + 1) \approx 0.667 = 66.7\%$$

Of all the transactions that really were fraud, the model caught about 66.7% (2 out of 3). The other third got through undetected.

(e) F1 Score

$$\text{F1} = 2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall}) = 2 \cdot (0.5 \cdot 0.667) / (0.5 + 0.667) \approx 0.571 = 57.1\%$$

The F1 score is the harmonic mean of precision and recall. At about 57.1%, it shows that when you look at both how trustworthy the fraud flags are and how much fraud gets caught, the model is only moderately good at finding fraud. That's much lower than the 85% accuracy suggests.

(f) False Positive Rate (FPR)

$$\text{FPR} = \text{FP} / (\text{FP} + \text{TN}) = 2 / (2 + 15) = 2/17 \approx 0.118 = 11.8\%$$

Of all the legitimate transactions, the model wrongly flagged about 11.8% as fraud. In practice, roughly 1 in 8.5 normal purchases would get blocked or sent for review.

Takeaway: 85% accuracy looks good, but it's misleading because the data is imbalanced (only 3 of the 20 transactions are fraud). A model that predicted "Not Fraud" for everything would still get $17/20 = 85\%$ accuracy while catching zero fraud. Precision, recall, and F1 tell you much more about how well the model actually detects fraud.

Exercise 3 – Iris Dataset with Logistic Regression

About the dataset: The Iris dataset has 150 flowers, 50 for each of 3 species: Iris setosa, Iris versicolor, and Iris virginica. Each flower has 4 features measured in cm: sepal length, sepal width, petal length, and petal width. The problem is that logistic regression is a binary classifier. It uses the sigmoid function $\sigma(\theta^T x) = 1 / (1 + e^{-(\theta^T x)})$ to give the probability that $y = 1$, and then uses a threshold (like 0.5) to pick between 2 classes. Since there are 3 species, I have to combine several binary logistic regression models. The two strategies I'd use are training one model per species, and a two-step approach. The UCI description also says one species is linearly separable from the other 2, but the other 2 are not linearly separable from each other. That species is Setosa, and the fact matters for strategy 2.

For both strategies, I'd first split the 150 flowers into a training set and a test set (for example 80% train and 20% test). Every model would be scikit-learn's `LogisticRegression()`, which learns θ by minimizing the average log loss with gradient descent–style methods. It also includes regularization through C (default $C = 1$, and a higher C means less regularization).

Strategy 1: One Model per Species

(a) Models to train: I'd train 3 binary logistic regression models, one per species:

- Model 1: Setosa ($y = 1$) vs. not Setosa (Versicolor or Virginica, $y = 0$)
- Model 2: Versicolor ($y = 1$) vs. not Versicolor ($y = 0$)
- Model 3: Virginica ($y = 1$) vs. not Virginica ($y = 0$)

Each model trains on the whole training set with all 4 features. The only difference is how the labels are set to 1 or 0: for each species, that species is 1 and the other two are 0. Each model learns its own θ , so its sigmoid output is the probability that a flower is that species.

(b) For a new flower, I'd plug its 4 measurements into all 3 models to get 3 probabilities (for example Setosa 0.02, Versicolor 0.70, Virginica 0.35) and predict the species with the highest one. I wouldn't use a 0.5 threshold on each model, because a flower could end up above 0.5 for two species or for none.

(c) I'd use accuracy on the test set, since the classes are balanced (50 of each species), unlike the fraud example. I'd also make a 3x3 confusion matrix to see which species get mixed up. I expect Versicolor and Virginica to be confused with each other, since Setosa is easy to separate. I'd also compute precision and recall for each species by treating it as the positive class. Finally, I'd compare training and test accuracy to check for overfitting or underfitting, and adjust C if needed.

(d) Benefits and drawbacks:

- **Benefits:** Simple. You only need 1 model per species (3 models). Every model trains on all the training data, which matters because 150 flowers is a small dataset. Each model's output is easy to understand: "the probability this flower is species X."
- **Drawbacks:** Each model trains on imbalanced data, because 1 species (50 flowers) is up against the other 2 combined (100 flowers). It's like the fraud example, just much less extreme. Lumping Versicolor and Virginica together as "not that species" can make it harder for a model to learn the boundary between those two, which is exactly the hard part of this problem. Also, the 3 models are trained separately, so their probabilities don't have to add up to 1, and two models can both be fairly confident.

Strategy 2: Two-Step Approach

(a) Models to train: I'd train 2 binary logistic regression models and use them one after the other:

- **Model A:** Setosa ($y = 1$) vs. not Setosa (Versicolor or Virginica, $y = 0$), trained on the whole training set.
- **Model B:** Versicolor ($y = 1$) vs. Virginica ($y = 0$), trained only on the Versicolor and Virginica flowers in the training set.

Both models use all 4 features. Model A separates Setosa from the other two species. I put Setosa first because UCI says it's the linearly separable species, so Model A should almost never get it wrong. Model B is trained only on Versicolor and Virginica, and its data is balanced (50 vs. 50 in the full dataset).

- (b) I'd plug a new flower into Model A first. If its output is 0.5 or higher, I predict Setosa and stop. Otherwise, I plug the flower into Model B: if its output is 0.5 or higher, I predict Versicolor, and if not, Virginica.
- (c) I'd measure performance the same way as strategy 1 so the two can be compared fairly: test accuracy, a 3x3 confusion matrix, precision and recall for each species, and a training vs. test comparison to check for overfitting or underfitting. I'd also check each model on its own. Model A's accuracy shows whether the first step is nearly perfect, and Model B's accuracy shows how well the hard pair is separated.

(d) Benefits and drawbacks:

- **Benefits:** Only 2 models instead of 3, and every flower only goes through 1 or 2 of them. Each model is a normal yes/no problem with a 0.5 threshold, just like in class. Model B's data is balanced and it focuses entirely on the hardest pair, Versicolor vs. Virginica, without Setosa getting in the way.
- **Drawbacks:** If Model A calls a Setosa "not Setosa," Model B has to pick Versicolor or Virginica, so the flower is guaranteed to be wrong. Model A's data is imbalanced (50 Setosa vs. 100 others), though that's not a big deal because Setosa is easy to separate. Model B only trains on about 80 flowers instead of 120, and less training data can raise variance, meaning the model is more sensitive to which flowers ended up in the training set and more likely to overfit. The order also had to be chosen by hand from what I know about the data, which wouldn't work as easily on a dataset with more classes.

Recommendation

For the Iris dataset, I'd use the two-step approach. UCI says Setosa is linearly separable from the other two species, so Model A should be almost perfect, and the drawback of mistakes carrying over barely applies. That leaves Model B, which spends all of its effort on Versicolor vs. Virginica, the only pair that's actually hard, using balanced data. Strategy 1 lumps those two species into "not X," which can blur that boundary. Model B does train on less data, so I'd use a learning curve to check it for overfitting and tune C if needed. If the confusion matrix showed the two-step approach doing worse than strategy 1, I'd switch to strategy 1.

CPSC 483 Homework 2

In this homework assignment, you'll explore the wine quality dataset from the UCI Machine Learning Repository.

LIST COLLABORATORS HERE (double click a text block to edit it):

Setup

This project requires Python 3.7 or above:

```
import sys

assert sys.version_info >= (3, 7)
```

Scikit-Learn 1.0.1 is required:

```
from packaging import version
import sklearn

assert version.parse(sklearn.__version__) >= version.parse("1.0.1")
```

Let's define the default font sizes, to plot pretty figures:

```
import matplotlib.pyplot as plt

plt.rc('font', size=12)
plt.rc('axes', labelsz=14, titlesz=14)
plt.rc('legend', fontsize=12)
plt.rc('xtick', labelsz=10)
plt.rc('ytick', labelsz=10)
```

Please choose a different random seed - feel free to match the seed with your collaborators to see if you get similar results.

```
import numpy as np
import pandas as pd
import time

seed = 10 # Change this to your favorite number
np.random.seed(seed)
```

We'll need to install the package first that allows you to download datasets from the UCI ML Repo through the `ucimlrepo` package, which is not pre-installed in Google Colab. You can do command line inputs by putting an exclamation point before your code line to install this package.

```
!pip install ucimlrepo
```

```
Requirement already satisfied: ucimlrepo in
/usr/local/lib/python3.13/dist-packages (0.0.7)
Requirement already satisfied: pandas>=1.0.0 in
/usr/local/lib/python3.13/dist-packages (from ucimlrepo) (2.2.3)
Requirement already satisfied: certifi>=2020.12.5 in
/usr/local/lib/python3.13/dist-packages (from ucimlrepo) (2026.7.22)
Requirement already satisfied: numpy>=1.26.0 in
/usr/local/lib/python3.13/dist-packages (from pandas>=1.0.0->ucimlrepo) (2.1.3)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.13/dist-packages (from pandas>=1.0.0->ucimlrepo)
(2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.13/dist-packages (from pandas>=1.0.0->ucimlrepo) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.13/dist-packages (from pandas>=1.0.0->ucimlrepo) (2026.3)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.13/dist-packages (from
python-dateutil>=2.8.2->pandas>=1.0.0->ucimlrepo) (1.17.0)
```

We will be working with the [Wine Quality](#) dataset, where the aim is to predict a wine's quality score (out of 10) based on its attributes.

I am providing you with the code to download the dataset, formatting it into a Pandas DataFrame, and doing some mild preprocessing, so there's nothing to change in this block (unless you want to - then knock yourself out!). But take a look to make sure you understand how I'm dealing with the data.

```
from ucimlrepo import fetch_ucirepo
from sklearn import set_config
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
```

```

# fetch dataset
wine = fetch_ucirepo(id=186)

# data (as pandas dataframes)
wine_df = wine.data.original

# Transform the color column to a binary is_red column for simplicity
wine_df.color = (wine_df.color == 'red').astype(int)
wine_df = wine_df.rename(columns = {'color': 'is_red'})

# Create a Test Set
train, test = train_test_split(wine_df, test_size=0.2, random_state=seed)

# Separate out the labels
wine_train = train.drop("quality", axis=1)
wine_train_labels = train['quality'].copy()

# Create preprocessing pipeline
num_pipeline = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
color_pipeline = make_pipeline(SimpleImputer(strategy="median"))
num_columns = wine_train.columns.drop("is_red")
preprocessing = ColumnTransformer([
    ("num_features", num_pipeline, num_columns),
    ("color", color_pipeline, ["is_red"])
])

# Peak at our preprocessed data
wine_processed = preprocessing.fit_transform(wine_train)
wine_processed_df = pd.DataFrame(
    wine_processed,
    columns=preprocessing.get_feature_names_out(),
    index=wine_train.index)
wine_processed_df.head()

```

	num_features__fixed_acidity	num_features__volatile_acidity	num_features__citric_acid	num_f
2705	-1.543902	-0.601460	0.898514	-0.3496
2120	0.526591	-0.907729	-0.406451	-0.7073
2145	-0.700368	-0.478952	-0.543816	0.36580
2756	0.449906	-0.233936	-0.200404	2.3964
398	3.287249	1.542426	1.860066	-0.6021

Part 1 - Creating Custom Linear Regression Solvers

We discussed in class how scikit-learn only supports two ways to fit a linear regression model. In this task, you will implement the other three methods mentioned in class to fit linear regression models by creating custom classes.

Exercise 1 - Using the Normal Equations

Use NumPy's matrix manipulation library for **ridge** regression using the normal equations.

Remember to include a bias column in the 0th column by making a copy of the original X and adding the column to X . The functions `np.ones`, `np.hstack`, and `np.eye` may be helpful for your implementation (see the NumPy docs). Also note that `@` is for matrix multiplication, while `*` is element-wise multiplication / dot product!

```
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.model_selection import train_test_split

class LinearRegressionNormalEquation(BaseEstimator, ClassifierMixin):
    def __init__(self, alpha=0):
        self.theta_ = None # This is your model parameters to learn
        self.alpha = alpha # For ridge regression

    def fit(self, X, y):
        # Types should be NumPy arrays
        X = np.asarray(X, dtype=float)
        y = np.asarray(y, dtype=float)
        m, n = X.shape

        # Put your normal equation process here!
        X_b = np.hstack([np.ones((m, 1)), X.copy()])

        # Ridge penalty matrix; the bias term (0th entry) is not regularized
        A = np.eye(n + 1)
        A[0, 0] = 0

        #  $\theta = (X^T X + \alpha * A)^{-1} X^T y$ 
        theta = np.linalg.solve(X_b.T @ X_b + self.alpha * A, X_b.T @ y)

        # Save your learned theta_ value
        self.theta_ = theta

        return self

    def predict(self, X):
        # Put how you predict with the learned self.theta_ here
        X = np.asarray(X, dtype=float)
```

```
X_b = np.hstack([np.ones((X.shape[0], 1)), X])

return X_b @ self.theta_ # Should return an np.ndarray of predictions
```

You should then be able to run the following code if the above is implemented properly, which runs a 5-fold cross validation to compute the R^2 , along with the runtime needed to do cross validation (don't worry about the actual R^2 value yet. To check your work, you should have an average R^2 somewhere between 0.2 and 0.4).

```
from sklearn.model_selection import cross_val_score

start_time = time.perf_counter()

normaleq = make_pipeline(preprocessing, LinearRegressionNormalEquation(alpha =
0.001))

normal_r2 = cross_val_score(normaleq, wine_train, wine_train_labels,
                           scoring="r2", cv=5)

end_time = time.perf_counter()
runtime_normaleq = end_time - start_time
pd.Series(normal_r2).describe()
```

	0
count	5.000000
mean	0.290602
std	0.030757
min	0.260718
25%	0.267201
50%	0.291166
75%	0.295130
max	0.338796

Exercise 2 - Batch Gradient Descent

Implement batch gradient descent for **elastic-net** regression. Have the stopping criterion be when $\|g(\theta)\| < 0.0001$ **or** at 1000 epochs, whichever comes first.

Note that you may get odd results - one possible reason may be due to the learning rate. Try playing around with the learning rate - I have set it to 0.01 but see what works.

Hint: Some useful functions here may include `np.sign` for finding the sign of an array and `np.zeros` for initializing an array of zeros.

```

from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.model_selection import train_test_split

class LinearRegressionBatchGD(BaseEstimator, ClassifierMixin):
    def __init__(self, alpha=0, r=0.15, learning_rate=0.1):
        self.theta_ = None # This is your model parameters to learn
        self.alpha = alpha
        self.r = r # The balance between the two estimators
        self.learning_rate = learning_rate # eta

    def fit(self, X, y):
        # Type Check
        X = np.asarray(X, dtype=float)
        y = np.asarray(y, dtype=float)
        m, n = X.shape

        # Put your batch gradient descent process here!
        X_b = np.hstack([np.ones((m, 1)), X.copy()])
        theta = np.zeros(n + 1)

        for epoch in range(1000):
            gradient = 2 / m * X_b.T @ (X_b @ theta - y)

            # Elastic-net subgradient:

            penalty = 2 * self.r * self.alpha * np.sign(theta) + 2 * (1 - self.r)
* self.alpha / m * theta
            penalty[0] = 0
            gradient += penalty

            if np.linalg.norm(gradient) < 0.0001:
                break

            theta = theta - self.learning_rate * gradient

        # Save your learned theta_ value
        self.theta_ = theta

        return self

    def predict(self, X):
        # Put how you predict with the learned self.theta_ here
        # Should be the same as before
        X = np.asarray(X, dtype=float)
        X_b = np.hstack([np.ones((X.shape[0], 1)), X])

        return X_b @ self.theta_ # Should return an np.ndarray of predictions

```

You should then be able to run the following code if the above is implemented properly, which runs a 5-fold cross validation to compute the R^2 , along with the runtime needed to do cross validation. Note that the results should be similar (but not necessarily the exact same) between exercises 1 and 2.

```
start_time = time.perf_counter()

batch = make_pipeline(preprocessing, LinearRegressionBatchGD(alpha = 0.001))

batch_r2 = cross_val_score(batch, wine_train, wine_train_labels,
                           scoring="r2", cv=5)

end_time = time.perf_counter()
runtime_batch = end_time - start_time
pd.Series(batch_r2).describe()
```

	0
count	5.000000
mean	0.290604
std	0.030789
min	0.260519
25%	0.267242
50%	0.291299
75%	0.295196
max	0.338764

Exercise 3 - Mini-batch Gradient Descent

You should now adapt the code you already wrote in Exercise 2 to do mini-batch gradient descent. Use a batch size of 64 (i.e., 64 instances per batch). Feel free to change this though - the typical standard in machine learning is to set a batch size equal to powers of 2.

Ensure that your X and y values are shuffled together so that the labels stay with their corresponding instances. One way to do this is to shuffle indexes instead of the instances themselves using NumPy:

- First, initialize your rng `rng = np.random.default_rng(seed=seed)`
- Then, for each epoch, you run `indices = rng.permutation(m)` # m is the number of instances

Ensure that you are using a learning schedule to stabilize your subgradient steps over iterations and epochs. A common learning schedule is to start with initial parameters η_0 and η_1 , and have the schedule

$$\eta = \frac{\eta_0}{iteration + \eta_1}$$

where *iteration* refers to each minibatch update to θ .

Lastly, instead of stopping at certain magnitudes of the gradient (which we don't have the entirety of in mini-batch gradient descent), stop if the magnitude of the difference between the old θ and new θ is less than 0.0001 at the **end** of each epoch or if 1000 epochs have passed.

Note: Your implementation may be slower than `SGDRegressor` in scikit-learn, even for a batch size of 1 (which is the same as stochastic gradient descent). This is because scikit-learn has its own optimization strategies to make stochastic gradient descent faster, so don't sweat if your implementation is slower. But, your implementation should still be close to the optimal value of θ .

```
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.model_selection import train_test_split

class LinearRegressionMiniBatchGD(BaseEstimator, ClassifierMixin):
    def __init__(self, alpha=0, r=0.15, batch_size=64, epochs=1000, eta0=5,
eta1=50):
        self.theta_ = None # This is your model parameters to learn
        self.alpha = alpha
        self.r = r # The balance between the two estimators
        self.eta0 = eta0
        self.eta1 = eta1
        self.batch_size = batch_size
        self.epochs = epochs

    def fit(self, X, y):
        X = np.asarray(X, dtype=float)
        y = np.asarray(y, dtype=float)
        m, n = X.shape

        # Put your mini-batch gradient descent algorithm here!
        X_b = np.hstack([np.ones((m, 1)), X.copy()])
        theta = np.zeros(n + 1)
        rng = np.random.default_rng(seed=seed)
        iteration = 0

        for epoch in range(self.epochs):
            theta_old = theta.copy()
            indices = rng.permutation(m)

            for start in range(0, m, self.batch_size):
                batch_idx = indices[start:start + self.batch_size]
                X_batch = X_b[batch_idx]
                y_batch = y[batch_idx]
                batch_m = len(batch_idx)

                gradient = 2 / batch_m * X_batch.T @ (X_batch @ theta - y_batch)

                penalty = 2 * self.r * self.alpha * np.sign(theta) + 2 * (1 -
self.r) * self.alpha / m * theta
                penalty[0] = 0
```

```

        gradient += penalty

        # Learning schedule: eta = eta0 / (iteration + eta1)
        eta = self.eta0 / (iteration + self.eta1)
        theta = theta - eta * gradient
        iteration += 1

    if np.linalg.norm(theta - theta_old) < 0.0001:
        break

    # Save your learned theta_ value
    self.theta_ = theta

    return self

def predict(self, X):
    # Put how you predict with the learned self.theta_ here
    # Should be the same as before
    X = np.asarray(X, dtype=float)
    X_b = np.hstack([np.ones((X.shape[0], 1)), X])

    return X_b @ self.theta_ # Should return an np.ndarray of predictions

```

Again, you should check your implementation with the following code.

```

start_time = time.perf_counter()

minibatch = make_pipeline(preprocessing, LinearRegressionMiniBatchGD(alpha =
0.001))

minibatch_r2 = cross_val_score(minibatch, wine_train, wine_train_labels,
                                scoring="r2", cv=5)

end_time = time.perf_counter()
runtime_minibatch = end_time - start_time
pd.Series(minibatch_r2).describe()

```

	0
count	5.000000
mean	0.289566
std	0.031553
min	0.256027
25%	0.267134
50%	0.293037
75%	0.293860

	0
max	0.337774

Exercise 4 - Tuning a Linear Regression Solver Method

I have implemented `Ridge()` and `SGDRegressor()` to see the performance of the singular value decomposition and stochastic gradient descent methods for solving linear regression, and you can now compare the runtimes of each method on the wine quality dataset.

```
from sklearn.linear_model import Ridge, SGDRegressor

# SVD with Ridge Regression
start_time = time.perf_counter()

svd = make_pipeline(preprocessing, Ridge(alpha = 0.0001, solver="svd"))
svd_r2 = cross_val_score(svd, wine_train, wine_train_labels,
                        scoring="r2", cv=5)
end_time = time.perf_counter()
runtime_svd = end_time - start_time

# SGD for ElasticNet Regression
start_time = time.perf_counter()

sgd = make_pipeline(preprocessing, SGDRegressor(penalty = "elasticnet", alpha =
0.001, l1_ratio = 0.15))
sgd_r2 = cross_val_score(sgd, wine_train, wine_train_labels,
                        scoring="r2", cv=5)
end_time = time.perf_counter()
runtime_sgd = end_time - start_time
```

Prints out a summary of all their runtimes on 5-fold cross validation. Remember: each method's computational efficiency isn't the only factor to consider - ensure that you choose the right method by considering all benefits/drawbacks of each.

```
print("Runtimes for 5-fold cross validation")
print("Normal Equations: ", runtime_normaleq)
print("SVD: ", runtime_svd)
print("Batch Gradient Descent: ", runtime_batch)
print("Stochastic Gradient Descent: ", runtime_sgd)
print("Minibatch Gradient Descent: ", runtime_minibatch)
```

```
Runtimes for 5-fold cross validation
Normal Equations: 0.25723325399985697
SVD: 0.523095253999827
Batch Gradient Descent: 0.7520855269999629
```

Stochastic Gradient Descent: 0.6314099950000127
Minibatch Gradient Descent: 8.96286288400006

Based on your knowledge of each method and what you see in the time efficiency of each method, make an informed choice of which solving method you want to use.

Here, use your chosen solver method to determine if you are overfitting or underfitting, which you can do with the information from the following code bit (change `mysolver` to your chosen solver).

```
from sklearn.metrics import r2_score

mysolver = make_pipeline(preprocessing, Ridge(alpha = 0.0001, solver="svd"))

mysolver.fit(wine_train, wine_train_labels)
mysolver.score(wine_train, wine_train_labels) # This prints out the training  $R^2$ 
```

0.29788570471387843

Do the following: * Determine if the R^2 error is due to overfitting or underfitting * Take the correct course of action based on your knowledge by either making [polynomial features](#) and increasing regularization parameters.

Use `GridSearchCV` to tune all of the hyperparameters of your model. You should consider whether the vanilla Linear Regression model is overfitting or underfitting to help you choose which hyperparameters to search.

Try to get your cross-validation R^2 to improve from your grid search. You will likely need to create a new pipeline to do this.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline

# Insert your grid search here!

poly_pipeline = Pipeline([
    ("preprocessing", preprocessing),
    ("poly", PolynomialFeatures(include_bias=False)),
    ("scaler", StandardScaler()),
    ("ridge", Ridge(solver="svd")),
])

param_grid = {
    "poly__degree": [1, 2, 3],
    "ridge__alpha": [0.01, 0.1, 1, 10, 100, 1000],
}

grid_search = GridSearchCV(poly_pipeline, param_grid, cv=5, scoring="r2")
grid_search.fit(wine_train, wine_train_labels)
```

```
print("Best parameters:", grid_search.best_params_)
print("Best cross-validation R^2:", grid_search.best_score_)
print("Baseline cross-validation R^2 (SVD, no polynomial features):",
      svd_r2.mean())
```

```
Best parameters: {'poly__degree': 2, 'ridge__alpha': 100}
Best cross-validation R^2: 0.3254610720814239
Baseline cross-validation R^2 (SVD, no polynomial features): 0.29060212676565855
```

At the top of the next bit of code, set the first variable to your best estimator from grid search and find out how well you do in testing once you're satisfied.

Note: You may have only seen a modest improvement in R^2 (around the order of 0.01-0.1), and that's okay! It may be that linear/polynomial regression is not the best model after all for this particular dataset and/or the features are not predictive, which is quite possible since this dataset has not been well explored like some others we've seen so far. As a machine learning engineer, at this stage you may have had to go back and investigating your data a bit more.

```
from sklearn.metrics import r2_score

# Insert your best estimator here!
final_model = grid_search.best_estimator_

wine_test = test.drop("quality", axis=1)
wine_test_labels = test['quality'].copy()

final_predictions = final_model.predict(wine_test)

final_r2 = r2_score(wine_test_labels, final_predictions)
final_r2
```

```
0.3435262254572832
```

Part 2 - Classification Metrics with Logistic Regression

We will be using the same dataset, but now changing the target label. We will now use this dataset to predict whether an instance is red or white wine.

Like before, I'll provide you with the data preprocessing here.

```
# Separate out the columns
wine_train = train.drop("is_red", axis=1)
wine_train_labels = train['is_red'].copy()

# All features are numeric now, so the preprocessing is a bit simpler
preprocessing = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
```

```
wine_processed = preprocessing.fit_transform(wine_train)
wine_processed_df = pd.DataFrame(
    wine_processed,
    columns=preprocessing.get_feature_names_out(),
    index=wine_train.index)
wine_processed_df.head()
```

	fixed_acidity	volatile_acidity	citric_acid	residual_sugar	chlorides	free_sulfur_dioxide	total_s
2705	-1.543902	-0.601460	0.898514	-0.349654	-0.829812	-0.544608	0.20637
2120	0.526591	-0.907729	-0.406451	-0.707385	-0.486077	-1.270297	0.47024
2145	-0.700368	-0.478952	-0.543816	0.365806	-0.486077	0.404370	0.98038
2756	0.449906	-0.233936	-0.200404	2.396451	-0.056407	-0.432963	0.32951
398	3.287249	1.542426	1.860066	-0.602170	0.888865	-0.991186	-1.1833

Exercise 5 - Plotting Precision and Recall

We will explore first-hand how to visualize the precision-recall curve on this dataset.

You should use scikit-learn's `LogisticRegression` class as well as the `precision_recall_curve` function in your implementation. The output should be the precision recall curve based on your **training data**.

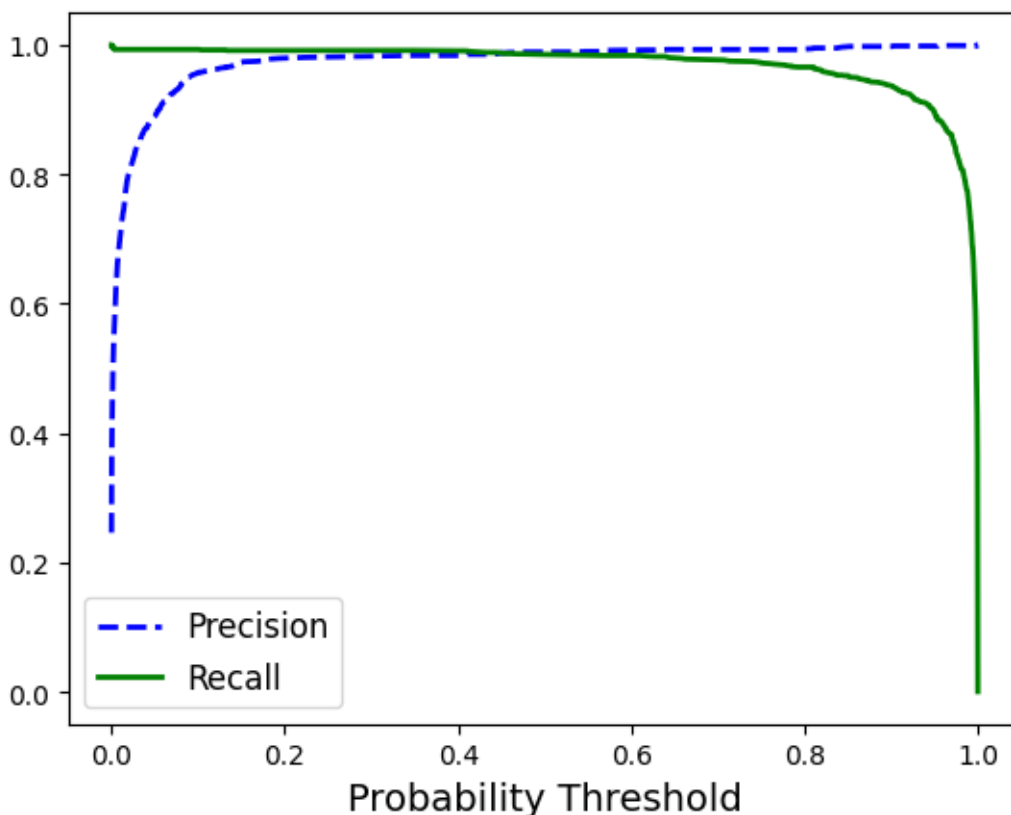
Read the documentation on both to learn how to use them properly (you do not need to adjust any default parameters for this exercise if you don't want to, but feel free to!)

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_recall_curve
import matplotlib.pyplot as plt

# Train your logistic regression model
log_reg = make_pipeline(preprocessing, LogisticRegression())
log_reg.fit(wine_train, wine_train_labels)

# Get the precisions, recalls, and thresholds from precision_recall_curve
y_probas = log_reg.predict_proba(wine_train)[: , 1]
precisions, recalls, thresholds = precision_recall_curve(wine_train_labels,
y_probas)

# Once you have your precisions, recalls, and thresholds, you can use this bit of
code
# to plot the precision and recall against thresholds
plt.plot(thresholds,precisions[:-1],"b--",label="Precision",linewidth=2)
plt.plot(thresholds,recalls[:-1],"g-",label="Recall",linewidth=2)
plt.xlabel("Probability Threshold")
plt.legend()
plt.show()
```



Although we are doing very well already (if all was implemented correctly), how can we hope to get the best F_1 score? The plot above should give you a hint of how to do this. (Answer in the text block below - which you can edit by double clicking it)

To get the best F_1 score, we should choose the decision threshold that maximizes F_1 instead of using the default 0.5. F_1 is the harmonic mean of precision and recall, so it is highest where the two are balanced, which is roughly where the precision and recall curves cross in the plot. We can compute $F_1 = 2PR / (P + R)$ for each threshold from `precision_recall_curve` and take the threshold with the highest value. Here, the best threshold is about 0.41 ($F_1 = 0.988$), compared to $F_1 = 0.987$ at 0.5. To avoid overfitting the threshold to the training set, it's better to choose it using cross-validated predictions (e.g., `cross_val_predict`).

Exercise 6 - The ROC

We will now explore the ROC curve.

Using the same trained Logistic Regression model from before, use the `RocCurveDisplay` class to display the ROC curve for your trained model. This time, say that the precision-recall was enough to convince you that vanilla out-of-the-box Logistic Regression was good enough, so we'll jump straight to testing and plotting the ROC curve on our testing set (probably not a good idea to do this in practice but humor me).

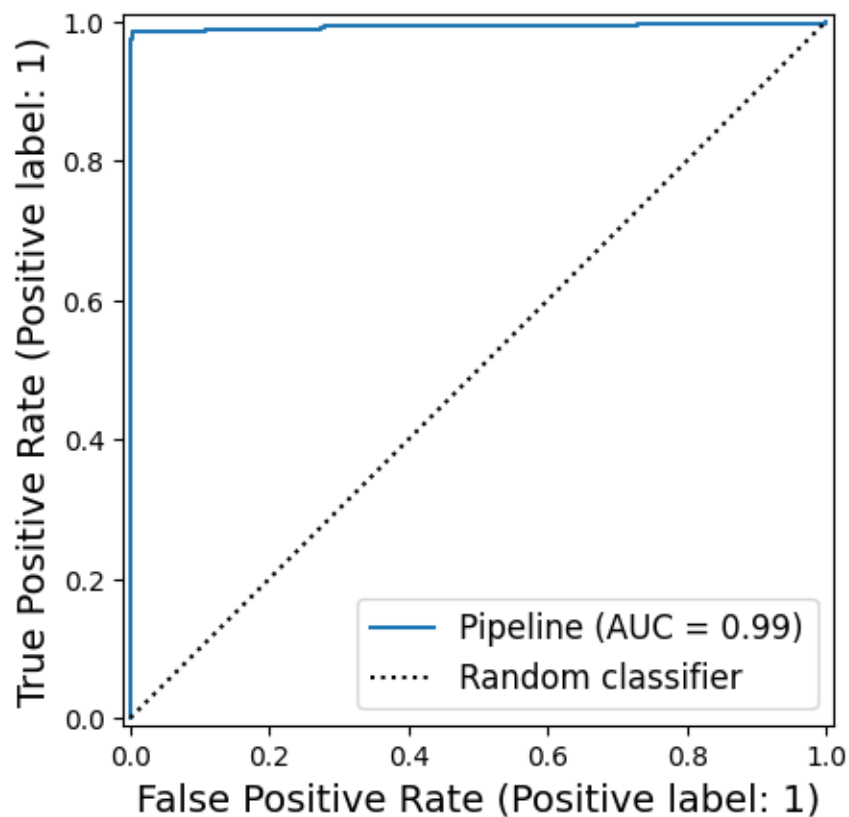
```

from sklearn.metrics import RocCurveDisplay

# Training Data
wine_test = test.drop("is_red", axis=1)
wine_test_labels = test['is_red'].copy()

# Your ROC curve display
RocCurveDisplay.from_estimator(log_reg, wine_test, wine_test_labels)
plt.plot([0, 1], [0, 1], "k:", label="Random classifier")
plt.legend()
plt.show()

```



The curve will display the AUROC value (shortened to AUC here). Write down your interpretation of the AUC in your own words in the context of the wine quality dataset.

The AUC on the test set is about 0.99. This means that if we pick one red wine and one white wine at random, the model gives the red wine a higher probability of being red about 99% of the time. An AUC of 0.5 is random guessing and 1.0 is perfect, so our model separates red from white wines almost perfectly. This is why the curve hugs the top-left corner: there is a threshold where we catch nearly all red wines while rarely mislabeling white wines as red.

Exercise 7 - Strength of Features in Determining Wine Type

Since our out-of-the-box logistic regression model is so good, let's look at which features have the highest $|\theta|$, as this can help indicate which feature(s) are the most influential in determining if a wine is red or white.

Print out the features from highest to lowest values of $|\theta|$, which will help us see which features are being heavily used in our model.

```
# Your solution to Exercise 7
theta = log_reg[-1].coef_[0]
feature_strengths = pd.DataFrame({
    "feature": wine_train.columns,
    "theta": theta,
    "|theta|": np.abs(theta),
}).sort_values("|theta|", ascending=False).reset_index(drop=True)

print("Intercept (bias):", log_reg[-1].intercept_[0])
feature_strengths
```

Intercept (bias): -4.167743093464917

	feature	theta	theta
0	density	3.463559	3.463559
1	residual_sugar	-3.419801	3.419801
2	total_sulfur_dioxide	-2.750840	2.750840
3	volatile_acidity	1.211128	1.211128
4	alcohol	1.118811	1.118811
5	chlorides	0.897397	0.897397
6	sulphates	0.719217	0.719217
7	free_sulfur_dioxide	0.715331	0.715331
8	citric_acid	-0.570021	0.570021
9	fixed_acidity	0.500109	0.500109
10	quality	0.307619	0.307619
11	pH	0.263912	0.263912

Based on your results, what are the top two most-used features used by your logistic regression model? Does a more positive value in those features correspond to the model having a higher change of labeling an instance as red or white wine?

The top two features are **density** ($\theta \approx +3.46$) and **residual sugar** ($\theta \approx -3.42$), followed by total sulfur dioxide ($\theta \approx -2.75$). Since the features are standardized, the sizes of the θ values are comparable, so these have the strongest influence on the prediction. Because the target is `is_red = 1`, a positive θ increases the log-odds of red and a negative θ increases the log-odds of white.

So yes, a more positive value in these features changes the prediction, but in opposite directions. A higher density makes the model more likely to label a wine as red, while a higher residual sugar

makes it more likely to label a wine as white. The sugar result fits the data, since white wines generally contain much more residual sugar than reds (about 6.4 g/L versus 2.5 g/L on average).